

result it should display the necessary date. In our case it would be 26.03.2013.

We will be using regular expressions and `sed` later on.

```
1. #!/bin/sh
2. DateVar='/bin/cat /usr/local/tomcat/webapps/ROOT/
   global_stats.html | /bin/sed -n ''$1/' '!{ x;d; }; '$1/'
   { x;x;p;n;n;p;x; }' | /bin/sed -n 's/.*<div>(.*)</
   div>.*^/1/p' | /bin/grep -v '$1';
3. echo "$DateVar";
```

Listing 2. You can see the regular expressions allow to finely cut relevant information only.

Line 1 shows that here comes an executable shell script. Line 2 is complex. Inside it there's a parameter `DateVar`, which will have the necessary date. Line 3 simply displays that date's value.

Let's look into Line 2 in more detail because it's a bit more complicated. How did all this magical transformation take place? First, the `cat` command prints `global_stats.html`, which we download from our HQ website. This file contains the entire range of goods that were finally imported into the corporate warehouse. Then, by means of pipe and after the first runwalk of the `sed` command, we filter out two fields—the item's ID and its date.

```
<div>00010000279379</div>
<div>26.03.2013</div>
```

Listing 3. Filtered out data after first runwalk of SED command.

Note that after the first SED pass (the following construction is used: `sed -n ''$1/' '!{ x;d; }; '$1/' { x;x;p;n;n;p;x; }'`), we will have data wrapped by HTML-code `<div>`, which will be cleared away during the second bypass. After further filtering by using the `sed` and `grep` commands, (`/bin/sed -n 's/.*<div>(.*)</div>.*^/1/p' | /bin/grep -v '$1'>`), we'll get pure data.

In this case, SED will remove `<div>` framing only, whilst GREP removes the whole row with the ID—we won't need it, because we only require a single date. Now, after feeding our script `get_hqdate.sh` with a parameter, i.e., the item's ID, we'll get the required date. Please note that the required date we receive is in the DD.MM.YYYY format, which differs from the usual PostgreSQL DATETIME format. It's quite obvious that our headquarters decided to ease the users' visual perception, and show dates in this format. However, it won't be very convenient to observe all three dates in different formats, so I would recommend that our readers think about how to convert the last date to the DATETIME format.

More details on SED usage and regular expressions can be found in the 'References' section.

So we have solved the first half of the problem. Now we need to insert this newly programmed shell-script into the "earlier-developed JSP-page (see July issue of OSFY, "Connecting All Data Sources Effectively, Part-1)" and make sure that everything works like a charm.

First of all, we need to enable file I/O operations inside our JSP-

Combined statistics

Details being fetched from Warehouse1, Warehouse2 online

Select Department	Department ID: 00010000050924
Select Department	
Operations Department, Prague	
Operations Department, Vienna	
Warehouse1, Vienna	
[1] ID: 00010000284721	
Allen Key	
Operations Department, Prague	
DELIVERY DATE TO WAREHOUSE1: 2013-03-15 14:00:40.456 (DELIVERED)	
DELIVERY DATE TO WAREHOUSE2: 2013-03-15 14:30:32.0 (DELIVERED)	
[2] ID: 00010000279379	
Flathead Screwdriver	
Operations Department, Prague	
DELIVERY DATE TO WAREHOUSE1: 2013-03-26 14:44:42.632 (DELIVERED)	
DELIVERY DATE TO WAREHOUSE2: 2013-03-26 15:44:32.0 (DELIVERED)	
[3] ID: 00010000282284	
Phillips Screwdriver	
Operations Department, Prague	
DELIVERY DATE TO WAREHOUSE1: 2013-03-15 13:59:02.72 (DELIVERED)	
DELIVERY DATE TO WAREHOUSE2: 2013-03-15 15:59:51.0 (DELIVERED)	
[4] ID: 00010000050945	
Towdler rail	
Operations Department, Prague	
DELIVERY DATE TO WAREHOUSE1: 2013-08-22 10:01:32.044 (DELIVERED)	
DELIVERY DATE TO WAREHOUSE2: 2013-08-22 10:59:51.0 (DELIVERED)	
[5] ID: 00010000000516	
Side frame & stretcher bar	
Operations Department, Prague	
DELIVERY DATE TO WAREHOUSE1: 2013-09-27 09:50:44.796 (DELAYED)	
DELIVERY DATE TO WAREHOUSE2: 2013-09-27 19:50:16.0 (NOT_DELIVERED)	

Figure 1: Combined statistics from two PostgreSQL databases

page. Otherwise, how could we access and call the shell-script? This is done at Line 1. Then we need to declare (Lines 2-3) a couple of string variables, one of which will be used to store the date for an item that finally arrived to the warehouse HQ.

```
1. <% page import="java.io.*,java.util.*"%>
2. <%! String HQ_Date = null; %>
3. <%! String hqDate = null; %>
```

Listing 4. You should add Java libraries for I/O and two new string variables.

Now let's proceed further and look at the following code snippet. It calls the above shell-script `get_hqdate.sh` and forms a string with the necessary date for unified statistics.

```
1. <c:set var="ROWID" value="${row_level1.ID}"></c:set>
2. <%
3.     String rowid = String.valueOf(pageContext.
   getAttribute("ROWID"));
4.     System.out.println("ROWID:" + rowid);
5.     String s = "/usr/local/tomcat/webapps/ROOT/get_hqdate.sh
   " + rowid;
6.     try {
7.         Process p = Runtime.getRuntime().exec(s);
8.         BufferedReader input = new BufferedReader(
   new InputStreamReader( p.getInputStream() ) );
9.         System.out.println("Runtime:" + s);
10.        while( ( HQ_Date = input.readLine() ) != null)
11.        {
12.            System.out.flush();
13.            hqDate = HQ_Date;
14.        }
```